Proposal for Labs and Projects

Contents

Human Resource Management System (HRM): Scenario and Requirements

Scenario: "Microservices-Oriented" HRM Platform The HRM system is a feature-rich internal human resource management platform designed to digitize the entire HR lifecycle (identity, attendance, leave, payroll). The platform requires absolute accuracy for financial data (payroll), High Availability, and a Real-time experience. Specifically, the system must scale well when the company size reaches thousands of employees through a distributed Microservices architecture.

Key Requirements

Type	Requirement Description	Architectural Driver
Functional	**Identity Management (Auth):** OTP Registration, JWT Login, RBAC Authorization (SUPERADMIN, MANAGER, STAFF), Department management.	Security, Distributed Identity Management.
Functional	**Leave Management (Leave):** Create requests, Multi-level approval, Leave balance calculation, Display work schedule matrix.	Logic processing performance, Data integrity.

Functional	**Attendance & Payroll (Salary/Attendance):** Record check-in/out, automatically calculate standard working days, penalize late/unexcused absences, finalize monthly payroll.	Large data processing (Batch Processing), Accuracy.
Functional	**Internal Communication (Noti):** Send system announcements, Real-time status updates for new/approved requests.	Asynchronous integration.
Non-Functional	**Performance & Real-time:** Any status change (request approval) must instantly show a Toast notification on the Admin/Manager's screen.	Event-Driven Architecture (EDA), Pub/Sub.
Non-Functional	**Availability & Fault Tolerance:** The database must survive even if 1 node goes down. The system should not hang if the Email service fails.	Redundancy (Postgres HA Patroni, Mongo Replica Set).
Non-Functional	**Scalability:** Clearly separate business domains to independently scale the Payroll service at the end of the month.	Microservices Decomposition, Load Balancing (Kong/Nginx).

Types of Software Architectures (For Comparison and Lab Use)

The HRM system is an excellent case study for comparing different architectural approaches.

1. **Layered Architecture (Monolithic):**

- **Description:** Initially, the system can be written entirely in a single backend directory with layers: Router (API) -> Controller -> Service -> Repository -> PostgreSQL.
- **Use Case:** Ideal for the MVP (Minimum Viable Product) phase to quickly validate HRM business logic.
- **Pros/Cons:** Easy initial development, but when the Payroll module runs heavy tasks, it will "hang" the Leave module as well.

2. **Microservices Architecture (Currently Applied):**
   - **Description:** The application is divided into independent auth_service, leave_service, salary_service, and noti_service. Uses Kong/Nginx as the API Gateway.
   - **Use Case:** Medium to large enterprise environments, where the IT team is divided into small squads managing each service.
   - **Pros:** Localized scalability (Scaling salary_service separately), polyglot persistence (PostgreSQL for core data, MongoDB for Logs).
3. **Event-Driven Architecture (EDA):**
   - **Description:** Communication via Message Broker (Redis). When there is a new Leave Request (leave_service), it "Publishes" an event to Redis, and noti_service "Subscribes" to send a Socket.io event.
   - **Use Case:** Background tasks like sending Emails (NodeMailer) or batch payroll processing.
   - **Pros:** Instant UI response (Non-blocking); if the Noti service crashes, Leave requests are still saved to the database normally.

---

8 Practical Labs (Labs)

The following 8 labs are designed closely around the HRM project's source code, guiding you from initial design to Microservices deployment and quality assessment.

Lab	Title	Architectural Focus	Activities & Detailed Step-by-Step Instructions
1	Requirements Elicitation	N/A (Pre-Architecture)	**Activity:** Identify Requirements for the HRM system.1. Identify actors: STAFF, MANAGER,

	& Modeling (Use Case)		SUPERADMIN.2. Identify Use cases: Attendance, Leave Approval, Payroll Finalization.3. Detail the "Monthly Payroll" use case.4. Identify Architecturally Significant Requirements (ASR): Must process 1000 employees concurrently without hanging the system.
2	Layered Architecture Design (Logical View)	Layered (Inside Service)	**Activity:** Design the architecture inside leave_service.1. Split the logic in the current leaves.js file into 3 layers: LeaveRouter, LeaveService, and LeaveRepository.2. Draw a UML Component diagram showing the flow from API request -> Business Logic (Check leave balance) -> Persistence (Save to PostgreSQL).
3	Layered Architecture Implementation (CRUD)	Layered	**Activity:** Refactor the leaves.js file.1. Separate SQL queries (like INSERT INTO leave_requests) from the Router.2. Move limit checking logic (max 5 people off/day, no more than 3 consecutive days) into the Business Logic layer.3. Implement a standard 3-layer POST /api/leaves API.
4	Microservices Decomposition & Communication	Microservices	**Activity:** Decompose the Monolith and establish API Contracts.1. Define boundaries for auth_service and leave_service.2. Determine how salary_service fetches attendance data from the DB (Via internal API instead of direct query).3. Draw a C4 Context Diagram connecting to NodeMailer.
5	Implementing the Salary Microservice	Microservices	**Activity:** Analyze the logic of the payroll.js file.1. Completely separate salary_service onto its own port (port 3000).2. Run as an independent container.3. Execute the syncPayrollRealtime function independently to connect to the database and calculate the monthly payroll grid.
6	API Gateway Implementation (Kong/Nginx)	Microservices	**Activity:** Configure the Gateway from the docker-compose.yml file.1. Configure Nginx on port 80.2. Forward /api/auth to auth_service and /api/leaves to leave_service.3. Understand how Kong API Gateway manages Rate Limiting and routing instead of Nginx.

7	Event-Driven Architecture (EDA) with Redis	Event-Driven	**Activity:** Implement Real-time Pub/Sub flow.1. Start the Redis container.2. When leaves.js calls POST /, instead of emitting Socket directly (hard to scale), write logic to Publish a "NEW_LEAVE" message to Redis.3. The noti_service Subscribes to Redis and executes io.emit to the browser.
8	Deployment View & ATAM Analysis	Deployment & Evaluation	**Activity:** Draw the physical diagram from docker-compose.yml.1. Draw nodes: Nginx, Kong, 4 Node.js Services, Postgres (Spilo/Patroni HA), Mongo Replica Set (rs0).2. ATAM Evaluation: Scenario if postgres-1 crashes, how does HAProxy switch to postgres-2? Scenario when needing to Scale salary_service at month-end.

Other Projects

Below are 4 supplementary projects from basic to intermediate, which can be used to practice skills before tackling the major project:

Project 1: To-Do List Application (Basic)

- **Scenario:** A personal tool to manage tasks. Add, view, mark as completed, delete (CRUD).
- **Architecture:** 1-Tier (Simple Client-Server).
- **Programming Steps (Node.js/Express):**
    - Create an in-memory array: let tasks = [{ id: 1, title: "Meeting", done: false }].
    - Write a POST /tasks API to push to the array. GET /tasks returns the list.

Project 2: Student Grade Management System (Intermediate)

- **Scenario:** Administrators add Courses, Students. Instructors enter grades.
- **Architecture:** Layered (2-Tier) with SQL.
- **Programming Steps (PostgreSQL & Node.js):**
    - SQL Schema: 3 tables Students, Courses, Enrollments.

    - Logic: Write an SQL JOIN function or API to calculate the weighted Grade Point Average (GPA).

Project 3: URL Shortener Service (Intermediate)

- **Scenario:** Like Bit.ly. Turn a long URL into a short code (e.g., hr.m/ab12), count clicks.
- **Architecture:** 3-Tier focusing on Read performance (Read-heavy).
- **Programming Steps:**
    - Use a Hash function or Base62 to convert an auto-incrementing ID into a character string.
    - GET /:code API queries the DB to find the original URL, increments click_count += 1, then returns res.redirect(301, long_url).

Project 4: Real-time Chat Application (Intermediate)

- **Scenario:** Department chat application. Messages appear instantly without F5 (refresh).
- **Architecture:** Event-Driven (WebSockets).
- **Programming Steps (React + Socket.io):**
    - Backend: io.on('connection') -> Listen to socket.on('chat_msg', msg) -> Broadcast io.emit('chat_msg', msg).
    - Frontend: Use useEffect to connect the Socket and update the UI State instantly.